

Backend Development & Server Architecture

Author / Lead: Shivam

Domain: Server Framework, RESTful API Routing, SSE Streaming, Web Scraping, Auth & Serverless Deployment

Core Technology: Node.js • Express • Pino • Cheerio • CORS • Crypto • Server-Sent Events (SSE) • Vercel Serverless

1. Executive Summary of Shivam's Work

Shivam engineered the server-side infrastructure, API gateway, request orchestration, and production deployment architecture of the **SatyaCheck** platform. Shivam's backend serves as the nerve center connecting the client application, web scraping pipelines, authentication, and the multi-tier AI inference engine.

Key Contributions & Engineering Achievements by Shivam:

- **Express Server Core (app.js & index.js):** Built an enterprise-grade Express application equipped with 50MB payload limits for base64 image ingestion, CORS middleware, structured Pino logging, and graceful process lifecycle termination.
- **REST API Gateway (routes/index.js):** Designed and implemented a modular router cleanly separating analytics, streaming chat, source credibility, viral trends, quiz generation, and authentication into distinct controllers.
- **Server-Sent Events (SSE) Streaming Engine (chat.js):** Designed real-time event-driven streaming enabling low-latency token delivery directly to the client browser without WebSocket connection overhead.
- **Automated Web Article Extraction (scraper.js):** Built an intelligent web scraper leveraging Cheerio and heuristics to extract clean article text, headline, author, and timestamp from raw news URLs.
- **Authentication & Session Management (auth.js):** Developed token-based authentication using cryptographically secure tokens (`crypto.randomBytes(32)`), credential validation, and privacy-first demo account fallbacks.
- **Cloud Serverless Transition (api/index.js & vercel.json):** Adapted the Express backend for zero-cold-start deployment on Vercel's serverless runtime, allowing the full stack to run anywhere without dedicated server costs.
- **Automated Deployment Automation (setup-and-run.ps1):** Created machine-independent PowerShell launcher scripts utilizing dynamic `$PSScriptRoot` path discovery.

2. Exact Codebase Files Authored & Owned by Shivam

The following table lists the core backend source files authored, maintained, and owned by Shivam:

File Path (Relative to Repository)	Module / Subsystem	Engineering Responsibilities & Implementation Details
artifacts/api-server/src/app.js	Express Application Factory	Express app setup, 50MB body parsers for base64 media, CORS, and Pino HTTP logger middleware.
artifacts/api-server/src/index.js	Server Entrypoint & Lifecycle	Environment variable validation, port binding (default 3000), SIGTERM/SIGINT signal listeners.
artifacts/api-server/src/routes/index.js	Master Router Aggregator	Registers and maps all sub-routers: /analyze, /chat, /credibility, /trending, /quiz, /auth, /healthz.
artifacts/api-server/src/routes/analyze.js	Analysis Controller	Coordinates content intake (text, URL, headline, image), triggers scraper, invokes AI, and saves results.
artifacts/api-server/src/routes/chat.js	SSE Chat Controller	Real-time Server-Sent Events (SSE) streaming endpoint with claim extraction and prompt assembly.
artifacts/api-server/src/routes/auth.js	Authentication Controller	User registration, login, Google OAuth handler, password reset, and cryptographic session tokens.
artifacts/api-server/src/routes/health.js	Health Probe Endpoint	Liveness check (GET /api/healthz) returning server timestamp, memory usage, and uptime.
artifacts/api-server/src/lib/scraper.js	Article Extraction Utility	Fetches external web pages, strips scripts/CSS, parses OpenGraph tags, and extracts article body.
artifacts/api-server/src/lib/logger.js	Structured Logging Utility	Configures high-speed Pino logger instance with formatted timestamps and log levels.
api/index.js	Vercel Serverless Gateway	Exports the Express application as a serverless handler for cloud deployment on Vercel.
setup-and-run.ps1	Developer Automation Script	One-click PowerShell bootstrap script handling dependency verification and server execution.

3. Shivam's Core Code Implementations

A. Server Setup & Middleware Configuration — `artifacts/api-server/src/app.js`

Shivam architected the Express instance with robust JSON parsing to support high-resolution base64 screenshot uploads:

```
// Author: Shivam (Backend Lead)
// File: artifacts/api-server/src/app.js
import express from "express";
import cors from "cors";
import pinoHttp from "pino-http";
import router from "../routes/index.js";
import { logger } from "../lib/logger.js";

const app = express();

// High-performance structured JSON logging
app.use(pinoHttp({ logger }));

// CORS configured to permit cross-origin calls from Vite dev server and production domains
app.use(cors());

// Parse incoming request bodies up to 50MB (required for base64 image analysis)
app.use(express.json({ limit: "50mb" }));
app.use(express.urlencoded({ extended: true, limit: "50mb" }));

// Route dispatch: Mount all API endpoints under /api
app.use("/api", router);

export default app;
```

B. Content Analysis & Ingestion Pipeline — `artifacts/api-server/src/routes/analyze.js`

Shivam created the primary fact-checking endpoint that intercepts URLs for live scraping before handing off to the AI service:

```

// Author: Shivam (Backend Lead)
// File: artifacts/api-server/src/routes/analyze.js
import { Router } from "express";
import { scrapeUrl } from "../lib/scrapper.js";
import { queryAI, runHeuristicFactCheck } from "../lib/ai-service.js";
import { db } from "../lib/db.js"; // Falls back to local storage if DB is offline

const router = Router();

router.post("/analyze", async (req, res) => {
  const { content, type, imageBase64 } = req.body;
  if (!content || !type) {
    return res.status(400).json({ message: "content and type are required" });
  }

  try {
    // 1. If type is URL, extract article text and publication metadata
    let articleContext;
    if (type === "url") {
      const scraped = await scrapeUrl(content);
      if (scraped.scrapedOk) {
        articleContext = [
          `Title: ${scraped.title}`,
          `Author: ${scraped.author}`,
          `Published: ${scraped.publishDate}`,
          `Body (${scraped.wordCount} words):\n${scraped.text.substring(0, 4000)}`,
        ].join("\n");
      }
    }

    // 2. Invoke multi-tier AI inference
    const result = await runAnalysis(content, type, imageBase64, articleContext);

    // 3. Persist record to persistent storage (PostgreSQL or local JSON storage)
    const savedRecord = await db.saveAnalysis({
      content: type === "image" ? "[Image Upload]" : content,
      sourceType: type,
      prediction: result?.prediction || "Unknown",
      confidence: Number(result?.confidence || 50),
      explanation: result?.explanation || "",
      keywords: result?.keywords || [],
      detectedSource: result?.detectedSource || null,
      factBreakdown: result?.factBreakdown || null,
    });

    return res.json({ ...savedRecord, ...result });
  } catch (err) {
    req.log?.error(err, "Analysis failed");
    return res.status(500).json({ message: "Failed to analyze content" });
  }
});

export default router;

```

C. Server-Sent Events (SSE) Streaming Engine — artifacts/api-server/src/routes/chat.js

Shivam engineered the streaming chat endpoint providing real-time progressive response tokens:

```
// Author: Shivam (Backend Lead)
// File: artifacts/api-server/src/routes/chat.js
router.post("/chat", async (req, res) => {
  const { message, conversationId } = req.body;

  // Establish Server-Sent Events protocol headers
  res.setHeader("Content-Type", "text/event-stream");
  res.setHeader("Cache-Control", "no-cache");
  res.setHeader("Connection", "keep-alive");

  const send = (event, data) =>
    res.write(`event: ${event}\ndata: ${JSON.stringify(data)}\n\n`);

  try {
    // 1. Check for domain guardrails (e.g. food recipes / off-topic content)
    if (isRecipeOrOffTopic(message)) {
      send("status", { type: "guardrail", message: "Redirecting query..." });
      send("token", { token: "🔍 **SatyaCheck Forensic Notice:**\nI am exclusively designed for forensic news verification. I do not provide cooking recipes or unrelated lifestyle content. Please submit news claims or viral headlines for fact-checking!" });
      send("done", {});
      return res.end();
    }

    // 2. Stream tokens in real-time
    send("status", { type: "analyzing", message: "Analyzing query..." });
    await streamChatResponse(message, (token) => {
      send("token", { token });
    });

    send("done", {});
    res.end();
  } catch (err) {
    send("error", { message: "Internal chat processing error" });
    res.end();
  }
});
```

D. Cloud Serverless Adapter — api/index.js

Shivam added the serverless cloud endpoint enabling zero-configuration Vercel deployment:

```
// Author: Shivam (Backend Lead)
// File: api/index.js
import app from "../artifacts/api-server/src/app.js";

// Export the configured Express app as a Vercel serverless function
export default app;
```

4. Verification & Testing Accomplished by Shivam

- **Payload Stress Testing:** Validated that 50MB base64 screenshot uploads parse smoothly without memory leaks.

- **SSE Stream Integrity:** Verified that event streams terminate cleanly with `res.end()` across client disconnects.
- **Scraper Fault Tolerance:** Tested URL extraction against malformed HTML, paywalled sites, and non-responsive domains with graceful fallback.

SatyaCheck Project Dossier • Lead Engineer: Shivam • Role 2: Backend Development